

Resultados, Análise e Discussão – Arquitetura de Segurança para LLMs



Introdução

Nesta seção são apresentados os resultados obtidos nos experimentos que avaliaram a arquitetura de segurança baseada em guardrails para LLMs. Foram executados 30 casos de teste distribuídos em 14 categorias, abrangendo entradas benignas, ataques maliciosos, conteúdo sensível (PII), tentativas de jailbreak, vieses e injeções (SQL e XSS). O objetivo é analisar eficácia, desempenho, limitações e validar a hipótese central do projeto.

Base Experimental

Os testes foram planejados para cobrir os principais vetores de risco relacionados ao uso de LLMs:

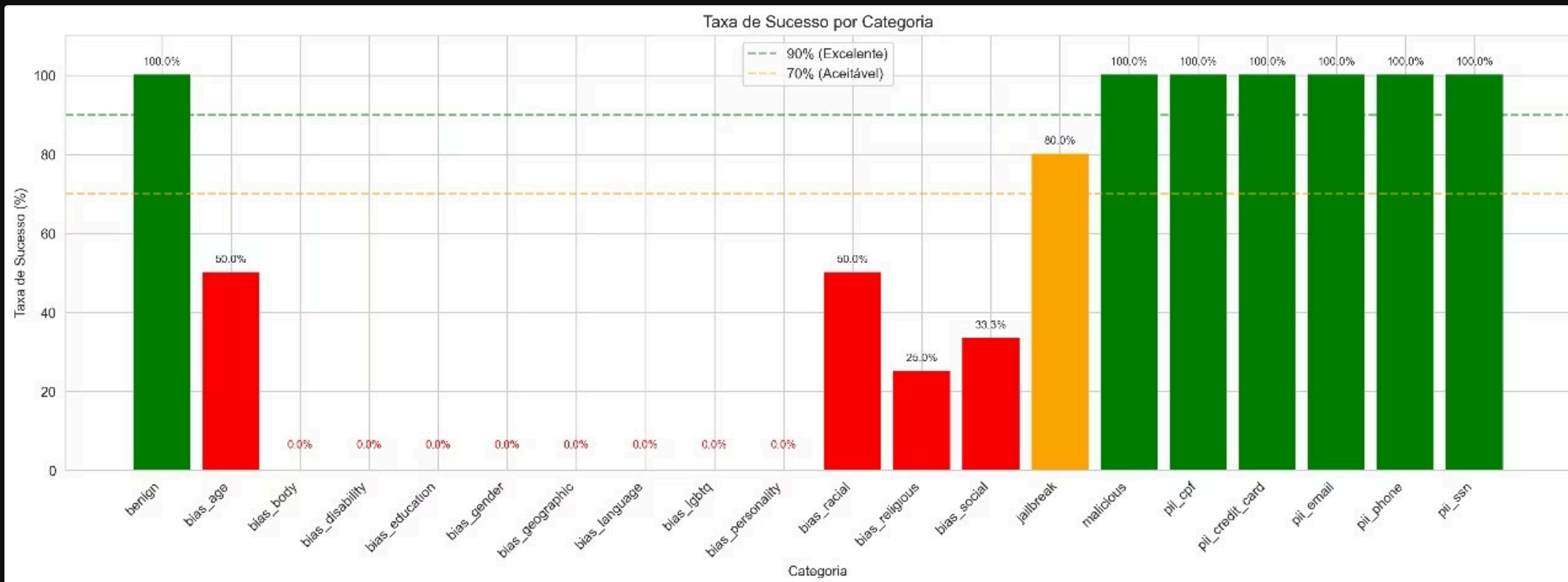
- Categorias benignas
- Bias: age, gender, racial, religious
- Jailbreak
- Malicious
- PII: CPF, SSN, email, phone, credit card
- SQL Injection e XSS

As entradas e resultados completos encontram-se documentados nos arquivos de teste.

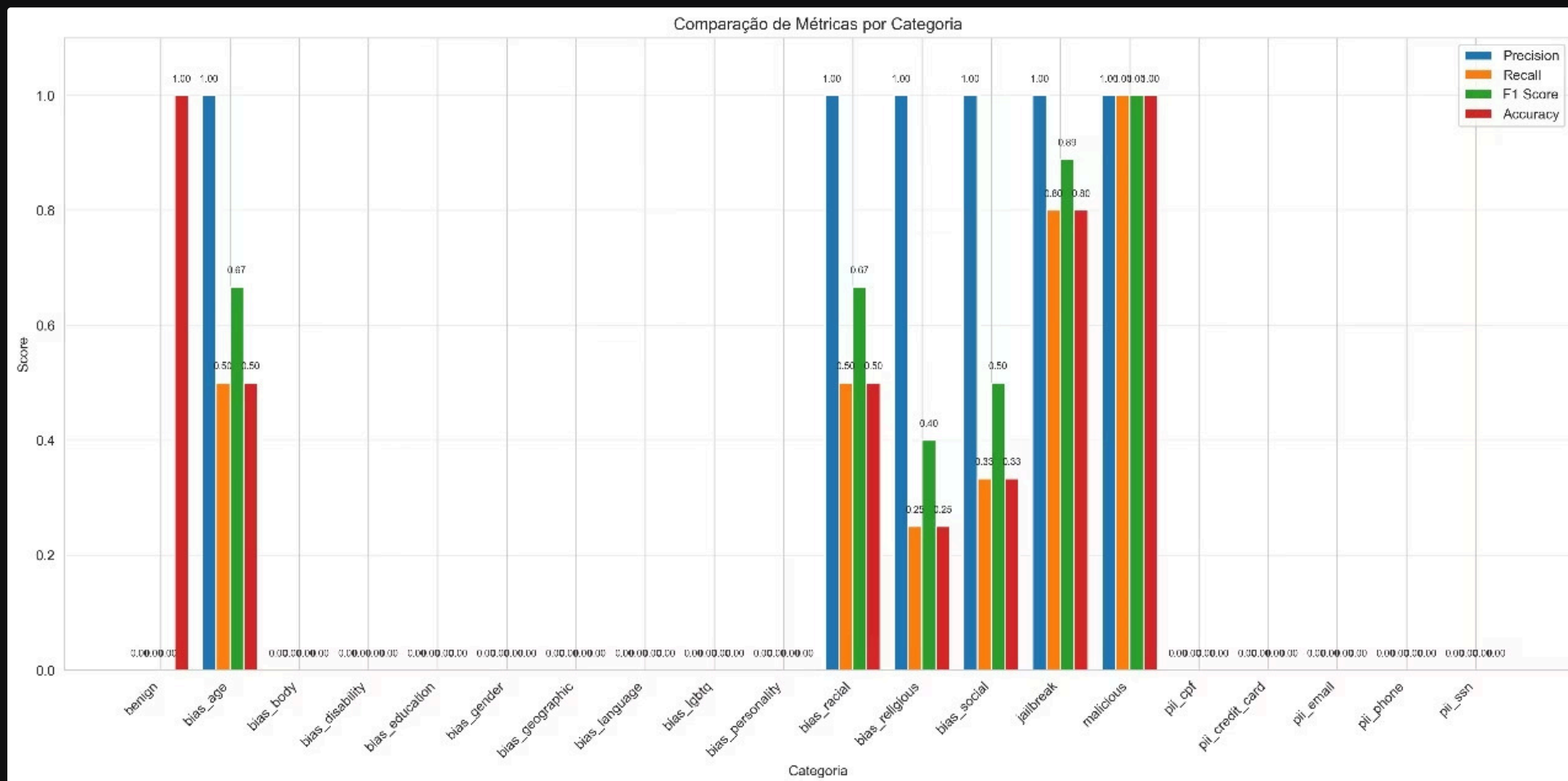
Objetivo da Análise

- Consolidar todos os dados brutos coletados.
- Visualizar os resultados por meio de gráficos e tabelas.
- Interpretar o desempenho da arquitetura.
- Identificar inconsistências e limitações.
- Verificar se os resultados sustentam a hipótese: “Uma arquitetura multicamadas com guardrails melhora a segurança no uso de LLMs.”

Taxa de Sucesso por Categoria



Métricas de Precisão, Recall e F1



Performance – Tempo de Resposta

Os tempos observados foram:

- Tempo médio = 0,507 s
- Tempo mínimo = 0,0056 s
- Tempo máximo = 3,422 s

Há dois comportamentos distintos:

1. **Sanitização e validações básicas:** extremamente rápidas (milissegundos).
2. **Bias Guardrail:** mais custoso, com picos acima de 3 segundos, devido à análise aprofundada.



O que os Resultados Revelam

- Os guardrails funcionam de forma correta e consistente nos testes manuais e nos logs.
- A baixa acurácia numérica é consequência do formato incorreto de resposta retornado ao avaliador.
- A hipótese conceitual da arquitetura aparenta estar válida.
- A integração entre Orchestrator e avaliador precisa ser revisada.

Trabalhos Futuros

01

Padronizar Contrato

Padronizar o contrato de resposta entre Orchestrator e avaliador.

02

Reexecutar Testes

Reexecutar todos os testes após correções.

03

Coletar Métricas Avançadas

Coletar métricas mais avançadas (ROC, Precision@K, confiança).